Personalized Networking Assistant

Project Description:

Personalized Networking Assistant is an AI-powered web application designed to help users build meaningful conversations during networking events. The system accepts an event description and the user's interests as input, extracts important discussion topics using DistilBERT, and generates personalized conversation starters using GPT-2. It also verifies factual information through the Wikipedia API and maintains conversation history and user feedback for future reference. The application is developed using FastAPI for backend services and Streamlit for the user interface, providing a simple, interactive, and efficient networking experience.

Scenarios

Scenario 1: A user enters an AI conference description and selects interests such as Machine Learning. The system generates personalized conversation starters related to AI and ML.

Scenario 2: A user attends a Cybersecurity seminar. The application extracts cybersecurity topics, verifies facts using Wikipedia, and generates relevant networking questions.

Scenario 3: After networking, the user saves the generated conversation history and submits feedback for future improvements.

Technical Architecture:

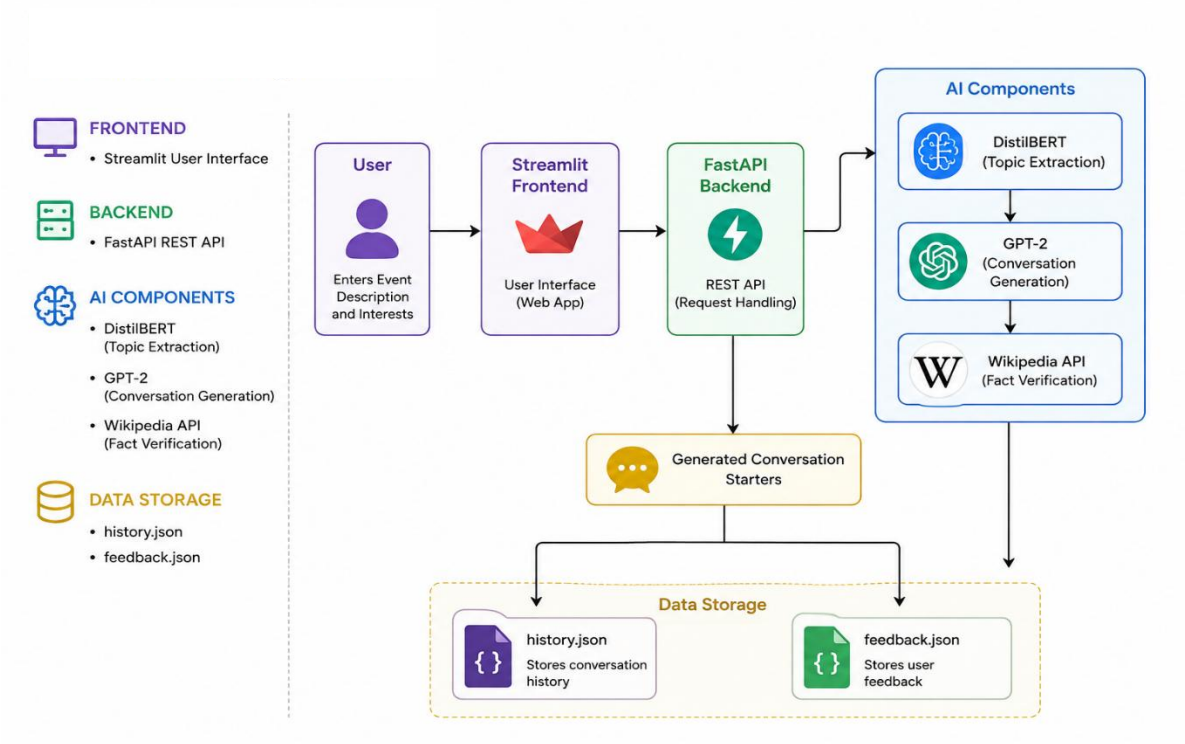


Figure 1: Technical Architecture of the Personalized Networking

Description: The Personalized Networking Assistant follows a modular architecture using a Streamlit frontend, FastAPI backend, DistilBERT for topic extraction, GPT-2 for conversation generation, and the Wikipedia API for fact verification. User interactions are stored in history.json and feedback.json files. The application runs locally and provides AI-powered networking conversation suggestions.

(Note: This project does not use a relational database. Therefore, an ER Diagram is not applicable (N/A).)

Pre-requisites:

- **Python 3.11+:** Python Official Documentation
- **FastAPI Framework:** FastAPI Documentation
- **Streamlit Framework:** Streamlit Documentation
- **Hugging Face Transformers:** Transformers Documentation
- **DistilBERT Model:**DistilBERT Documentation
- **GPT-2 Model:** GPT-2 Documentation
- **Wikipedia API:** Wikipedia-API Documentation
- **Version Control with Git:** Git Documentation
- **Development Environment Setup:** Visual Studio Code Documentation

Project Workflow:

Milestone 1: Model Selection and Architecture

- **Activity 1.1:** Select DistilBERT for topic extraction and GPT-2 for conversation generation.
- **Activity 1.2**: Design the overall architecture using Streamlit, FastAPI, AI models, and JSON storage.
- **Activity 1.3:** Configure the Python environment and install all required libraries.
- **Activity 1.4:** Verify the development environment and project structure.

Milestone 2: Core Functionalities Development

- **Activity 2.1:** Develop the Event Analyzer, Topic Generator, Fact Checker, and History Logger modules.
- **Activity 2.2:** Integrate FastAPI routes with the AI services and JSON data storage.

Milestone 3: Main Application Development

- **Activity 3.1:** Implement the main application logic and connect the frontend with backend services.

Milestone 4: Frontend Development

- **Activity 4.1:** Design the Streamlit user interface for event input and conversation generation.
- **Activity 4.2:** Display generated conversation starters, fact-check results, and conversation history.

Milestone 5: Testing and Local Deployment

- **Activity 5.1:** Perform unit testing using Pytest and verify API functionality.
- **Activity 5.2:** Run the application locally using FastAPI and Streamlit and validate all features.

Milestone 6: Conclusion

The Personalized Networking Assistant uses DistilBERT, GPT-2, FastAPI, Streamlit, and the Wikipedia API to generate personalized conversation starters for networking events. The project demonstrates the integration of AI models with a web application to provide an interactive and intelligent user experience.

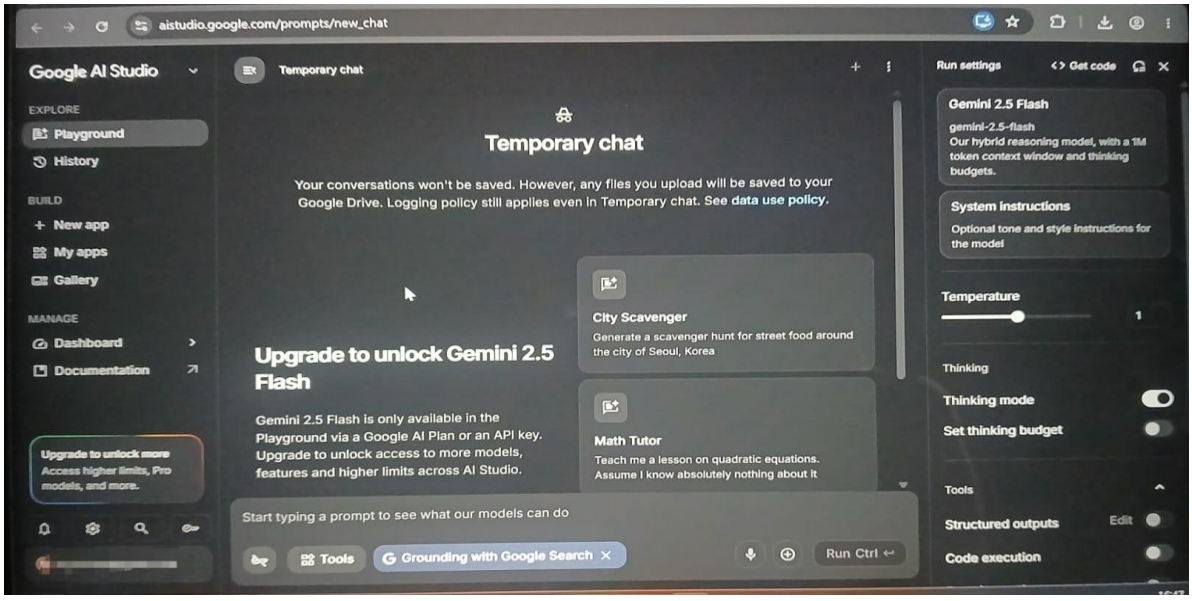
Milestone 1: Model Selection and Architecture

In this milestone, we select the AI technologies and define the overall architecture for the Personalized Networking Assistant. The project uses DistilBERT for topic extraction, GPT-2 for conversation generation, Wikipedia API for fact verification, FastAPI for backend services, and Streamlit for the user interface.

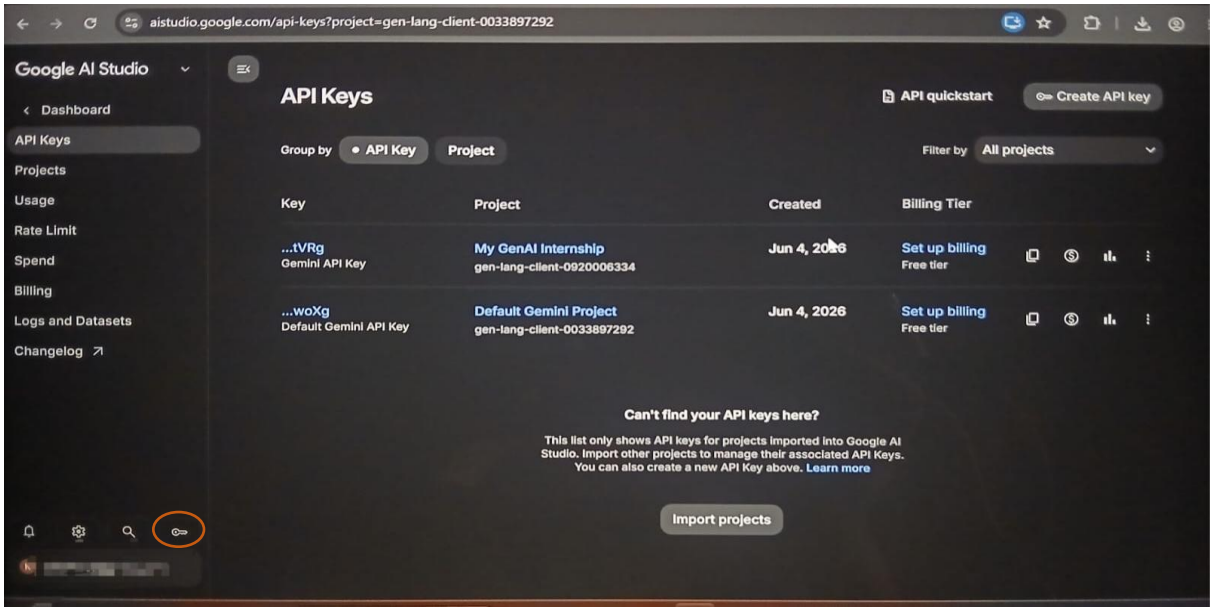
Activity 1.1: Configure Google Gemini API Key

Before the application can use AI services, a valid **Google Gemini API Key** is required. Follow the steps below to generate the API key and configure it securely in the project.

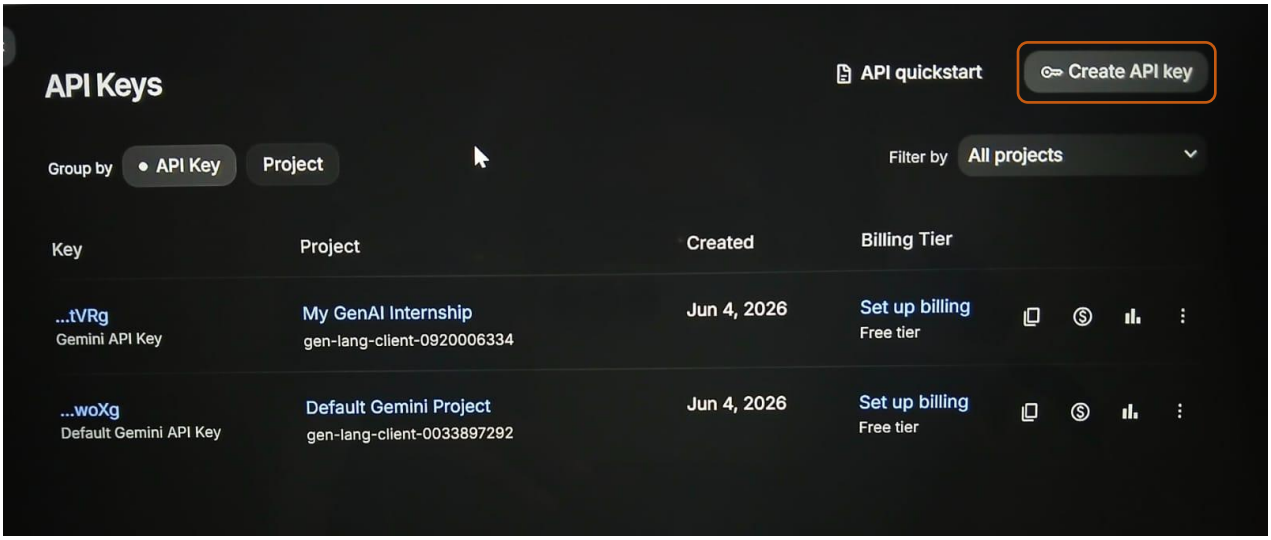
- **Step 1 — Open Google AI Studio**: Visit https://aistudio.google.com/ and sign in with your Google account.



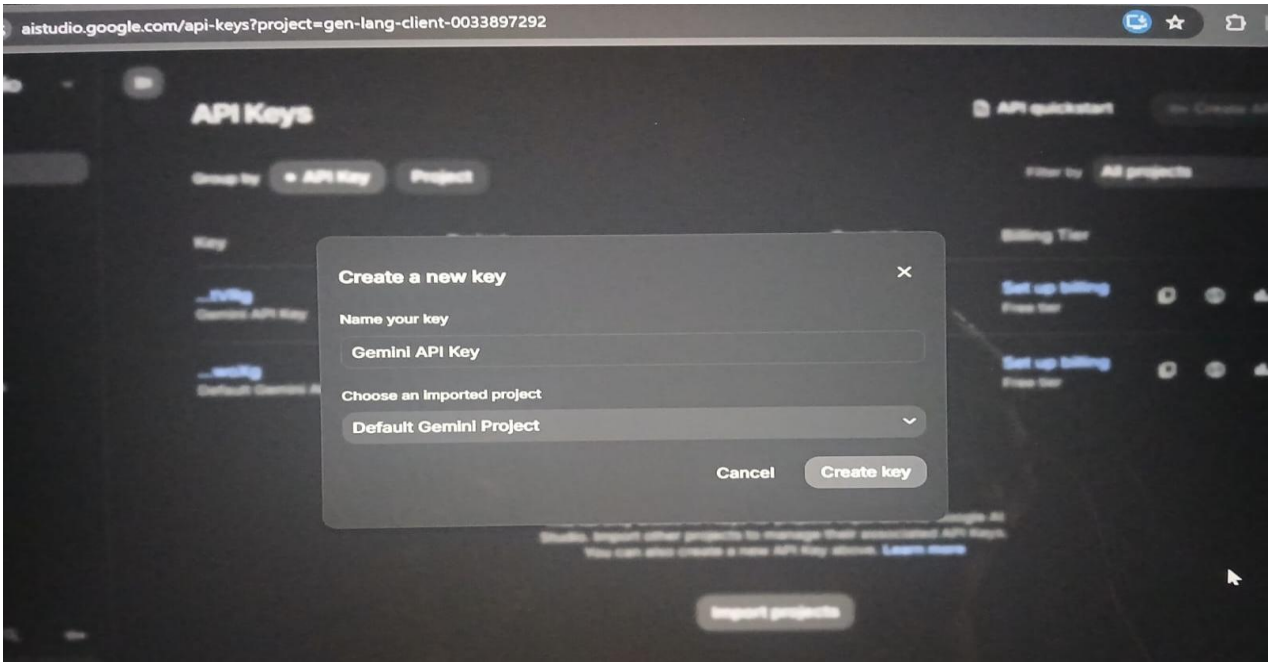
- **Step 2 — Open API Keys**: After signing in to Google AI Studio, click **Get API Key** or open the **API Keys** page from the Google AI Studio menu to access your API key management page.



- **Step 3 — Create a New API Key:** Click the "Create API Key" button. Enter a descriptive name (e.g., Personalized-Networking-Assistant-Key) and select your project. Then click **"Create Key"**.



- **Step 4 — API Key:** Enter a **Display Name** (e.g., *Networking Assistant API Key*). Leave **Expiration** as **No expiration** (optional), then click **Create key**. Copy the generated API key immediately and store it in a safe place because it will not be shown again after closing the dialog.



- **Step 5 — Add the Key to Your Project:** Create a .env file in the project folder and add your API key as shown below:

```
GOOGLE_API_KEY=your_api_key_here
```

- **Step 6 — Verify the Key is Loaded:** Load the API key in your Python application using the following code:

```
from dotenv import load_dotenv
import os
load_dotenv()
api_key = os.getenv("GOOGLE_API_KEY")
```

- **Important** — Never upload your API key to GitHub. Add the **.env** file to **.gitignore** to keep your key secure.

```
.env
```

Activity 1.2: Research and Select the Appropriate AI Models

Here are the points for selecting the best models for the project:

- **Understand the Project Requirements:** Review the needs of the Personalized Networking Assistant, including event analysis, topic extraction, conversation generation, and fact-checking.
- **Explore Model Documentation:** Study the documentation of DistilBERT, GPT-2, and the Wikipedia API to understand their features and limitations.
- **Evaluate Model Performance:** Compare models based on accuracy, response quality, processing speed, and suitability for networking conversations.
- **Select the Optimal Model:** Choose DistilBERT for topic extraction and GPT-2 for conversation generation because they provide good performance and fast responses.

Activity 1.3: Define the Architecture of the Application

- **Create an Architectural Diagram:** Design the architecture of the application, including Streamlit frontend, FastAPI backend, DistilBERT, GPT-2, Wikipedia API, and JSON storage.
- **Detail Frontend Functionality:** Describe how users enter event details and interests, generate conversation starters, and view conversation history.
- **Outline Backend Responsibilities:** Explain how the FastAPI backend processes user input, calls AI models and APIs, and manages data storage.

- **Describe AI Integration Points**: Define how DistilBERT extracts topics, GPT-2 generates conversation starters, and the Wikipedia API verifies information before returning results to the frontend.

Activity 1.4: Set Up the Development Environment

- **Install Python and Required Libraries:** Ensure Python 3.11+ is installed and install FastAPI, Streamlit, Transformers, Wikipedia API, Pytest, and other dependencies.
- **Create a Virtual Environment:** Create and activate a virtual environment to isolate project dependencies:

```
python -m venv venv

venv\Scripts\activate
```

- **Install FastAPI:** Use pip to install FastAPI and Uvicorn:

```
pip install fastapi uvicorn
```

- **Install Streamlit and Transformers:** Install Streamlit and Hugging Face Transformers:

```
pip install streamlit transformers
```

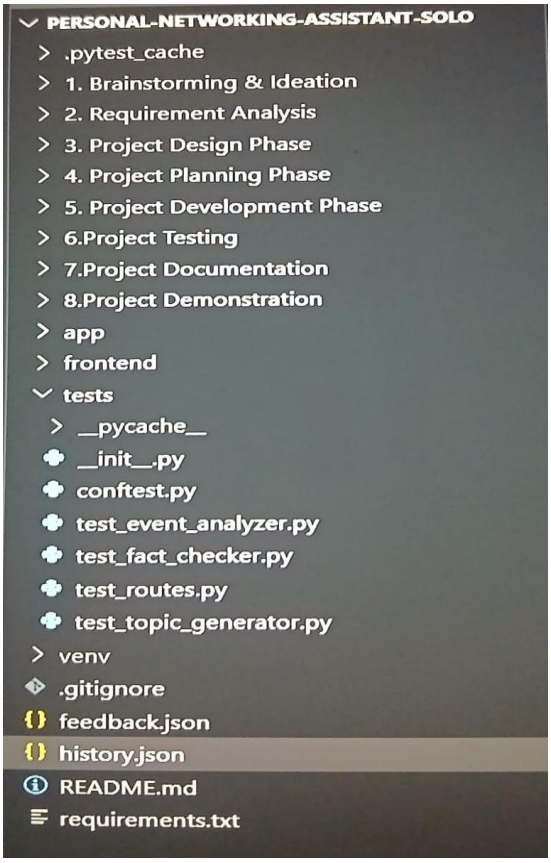
- **Install Additional Libraries:** Install the remaining libraries required for the project:

```
pip install wikipedia-api pytest httpx python-dotenv
```

- **Configure the API Key:** Create a **.env** file in the project folder and add:

```
GOOGLE_API_KEY=your_api_key_here
```

- **Set Up Application Structure:** Create the project structure containing the `app.py`, `requirements.txt`, `.env`, and other project files.



Milestone 2: Core Functionalities Development

Activity 2.1: Develop Core Networking Features

The application provides intelligent conversation starters for networking events using AI models.

- **Event Analyzer:** Extracts important topics and keywords from the event description using DistilBERT.
- **Topic Generator:** Generates two or three personalized conversation starters based on the event theme and user interests using GPT-2.
- **Fact Checker:** Verifies generated content using the Wikipedia API to improve reliability and accuracy.

Activity 2.2: Implement the FastAPI Backend

Define API Routes:

- Create API routes in **main.py** for theme extraction, conversation generation, fact-checking, and history management.

Process User Input:

- Create input forms in Streamlit for users to enter the event description and their interests.
- Use FastAPI request handlers to receive and process user input.
- Validate that the event description and user interests are not empty before processing.

Integrate AI Models:

- Call the Event Analyzer service to extract important topics using DistilBERT.
- Pass the extracted topics and user interests to the Topic Generator service, which uses GPT-2 to generate conversation starters.
- Use the Fact Checker service to verify the generated content through the Wikipedia API.
- Return the generated conversation starters and fact-check results to the frontend.

Milestone 3: App.py Development

Milestone 3 focuses on creating and refining the core logic of the app.py file, which serves as the backbone of the Personalized Networking Assistant. In this milestone, you'll set up each feature's functionality through FastAPI routes, establish reliable prompt construction and API interaction, and conduct unit testing to ensure each feature performs as expected.

Activity 3.1: Writing the Main Application Logic in app.py

Define the Core Routes in app.py:

- Establish routes for Personalized Networking Assistant endpoints:
  - / — Streamlit interface for entering event details and user interests.

```
import streamlit as st

st.title("Personalized Networking Assistant")

event_description = st.text_area("Enter Event Description")

user_interests = st.text_input("Enter Your Interests")

if st.button("Generate Conversation Starters"):
   st.write("Generating suggestions...")
```

  - **/generate** — API endpoint that analyzes event details and user interests and generates personalized conversation starters.

```
@app.post("/generate")

async def generate_conversation(event: EventRequest):

  event_description = event.event_description
  user_interests = event.user_interests

  topics = extract_topics(event_description)

  conversation_starters = generate_starters(
     topics,
     user_interests
  )

  fact_check_results = verify_with_wikipedia(topics)

  return {
     "success": True,
     "topics": topics,
     "conversation_starters": conversation_starters,
     "fact_check_results": fact_check_results
  }

except Exception as e:

  return {
     "success": False,
     "error": str(e)
  }
```

Set Up Route Handlers for Each Feature:

- For the /generate route, implement logic that reads event_description, user_interests, goal, and tone from the incoming JSON body using request.get_json().
- Apply input validation: return a 400 error if event_description is empty or exceeds the allowed limit.
- Route requests from the frontend to the /generate endpoint, which processes data and returns a JSON response.

Define Prompt Engineering Helpers:

- **TONE_DESCRIPTIONS** dictionary: maps professional, friendly, and warm tones to networking conversation styles used in the prompt.

```
TONE_DESCRIPTIONS = {
"professional": "formal, respectful, and business-appropriate.",
"friendly": "approachable, upbeat, and conversational.",
"warm": "personable and appreciative, suitable for follow-up conversations."
}
```

- **GOAL_DESCRIPTIONS** dictionary: maps networking goals to conversation guidance.

```
GOAL_DESCRIPTIONS = {
"cold_outreach": "a concise first message to someone new.",
"informational_interview": "a polite request to learn about a person's role.",
"post_event_follow_up": "a follow-up message after an event.",
"thank_you": "a short note of appreciation."
}
```

- **build_prompt()**: assembles a structured prompt using the event description, user interests, networking goal, and tone description.

```
def build_prompt(event_description: str, interests: str, goal: str, tone: str) -> str:

goal_desc = GOAL_DESCRIPTIONS.get(goal, "networking")

tone_desc = TONE_DESCRIPTIONS.get(tone, "professional")
```

Integrate the AI Responses in Each Function:

- Call client.chat.completions.create() Call the AI model with a system message and the assembled networking prompt.
- **extract_json()**: Use extract_json() to parse and validate the JSON response.

```
def extract_json(text: str) -> dict:
    """Extract JSON from LLM response, handling markdown code blocks."""
    # Try direct parse first
    try:
        return json.loads(text)
    except json.JSONDecodeError:
        pass
    # Try extracting from markdown code block
    match = re.search(r'```(?:json)?\s*([\s\S]*?)```', text)
    if match:
        try:
            return json.loads(match.group(1).strip())
        except json.JSONDecodeError:
            pass
    match = re.search(r'\{[\s\S]*\}', text)
    if match:
        try:
            return json.loads(match.group(0))
        except json.JSONDecodeError:
            pass
    raise ValueError("Could not extract valid JSON from response")
```

- Validate that the parsed result contains the required keys — message, icebreaker, follow_up, and facts.
- **Home route:** / → renders the main interface.
  /generate (POST) → accepts JSON and returns networking suggestions.

Milestone 4: Frontend Development

Milestone 4 focuses on developing a user-friendly web interface for the Personalized Networking Assistant. This includes creating responsive pages using HTML, CSS, and JavaScript to collect event details and user interests and display AI-generated conversation starters and networking suggestions.

Activity 4.1: Designing and Developing the User Interface

Set Up the Base HTML Structure:

- Develop the main HTML file (index.html) as the single-page interface, including a header with the Personalized Networking Assistant title, an input section for event details and user interests, and a results section.

- Use semantic HTML elements to keep the structure organized and ensure the interface is accessible for all users.

- Integrate Font Awesome icons for platform indicators and result section headings, and Google Fonts (Inter) for clean typography.

Design a Responsive Layout Using CSS:

- Create a CSS stylesheet (static/css/style.css) implementing a clean and responsive interface for the Personalized Networking Assistant.
- Use flexbox and grid layouts to organize event details, user inputs, and generated networking suggestions.
- Add media queries to ensure the layout adapts across desktop, tablet, and mobile screen sizes.

Create Separate UI Components for Each Output Type:

- **Conversation Cards**: display generated conversation starters in separate cards.
- **Theme Section**: display extracted themes and topics related to the event.
- **Fact-Check Section**: display verified information and networking suggestions.

Activity 4.2: Creating Dynamic Content Rendering with JavaScript

Handle Form Submission Asynchronously:

- Attach a submit event listener that sends event details and user interests as a JSON POST request to /generate.
- During processing, disable the submit button and show a loading indicator.

Render Results Dynamically:

- Implement the renderResults(data) function in main.js to dynamically display conversation starters, extracted themes, and fact-check results.
- After rendering, automatically scroll to the results section using scrollIntoView().

Restore UI State After Generation:

- In the finally block of the async handler, re-enable the submit button and restore the original button text after processing.

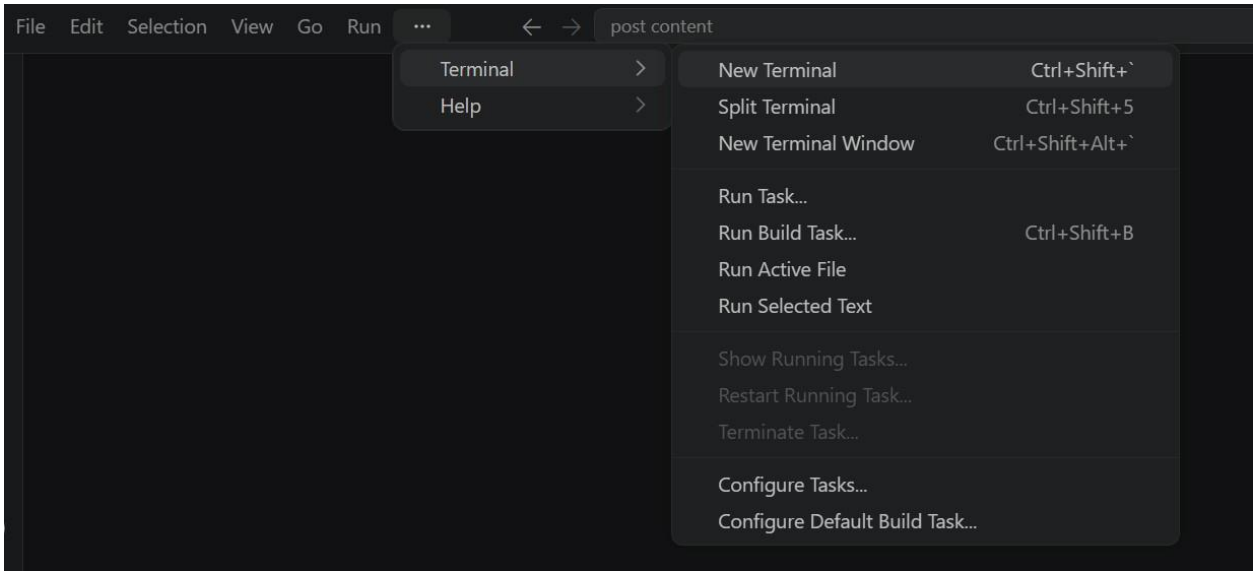
Milestone 5: Deployment

In Milestone 5, the focus is on deploying the Personalized Networking Assistant application locally and publicly. The application uses Streamlit as the frontend and FastAPI as the backend. Deployment includes configuring the environment, running the servers, and exposing the application using Ngrok.

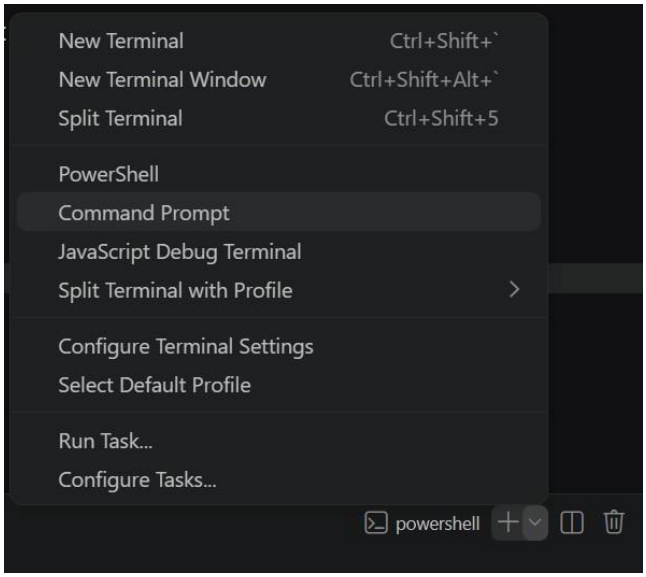
Activity 5.1: Preparing the Application for Local Deployment

Set Up a Virtual Environment:

- Open a New Terminal



- Then open “Command Prompt”



- Create and activate a virtual environment, then install all required dependencies from requirements.

```
python -m venv venv

venv\Scripts\activate

pip install -r requirements.txt
```

Configure Environment Variables:

- Create a .env file in the project root and add all required environment variables for the Personalized Networking Assistant.

Activity 5.2: Local Testing and Verification

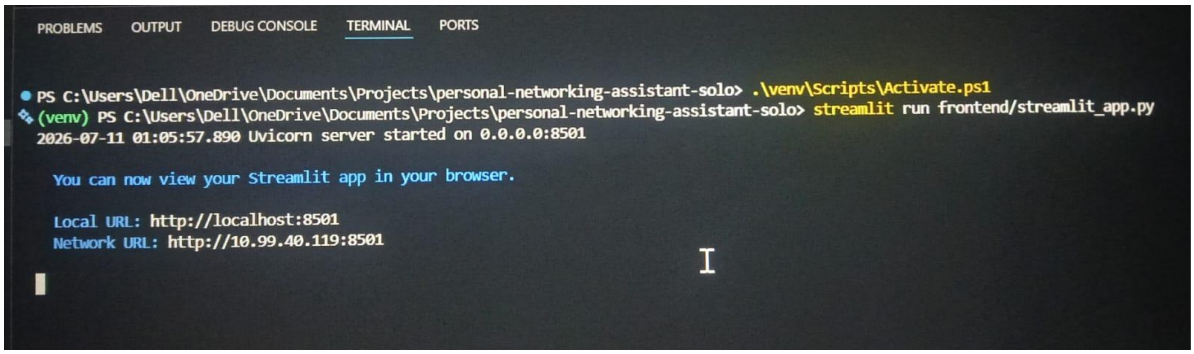
Start the Application::

- Run the application locally using:

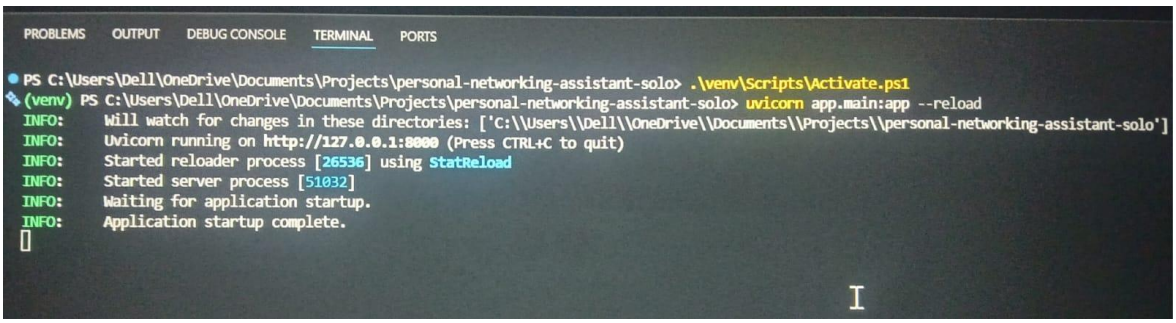
```
(venv) D:\projects>uvicorn app.main:app –reload

(venv) D:\projects>streamlit run app.py
```

- Once both servers are running, open a browser and navigate to:
- Frontend: http://localhost:8501



- Backend API: http://127.0.0.1:8000/docs



- Interact with the Personalized Networking Assistant by entering an event description and user interests. Verify that conversation starters and fact-check results are generated correctly.

Activity 5.3: Running the Application Locally

The Personalized Networking Assistant can be run locally using FastAPI for the backend and Streamlit for the frontend. Both services must be started to test the application.

Start Frontend and Backend Services:

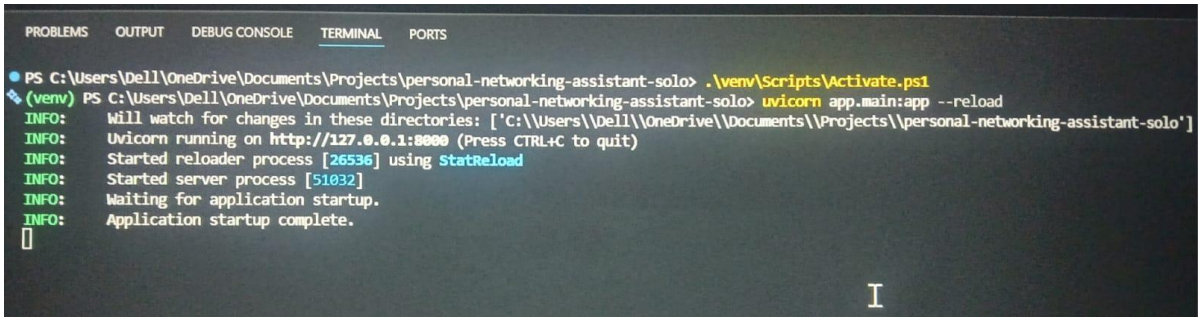
- Start the FastAPI backend server using the following command:
- `uvicorn app.main:app --reload`

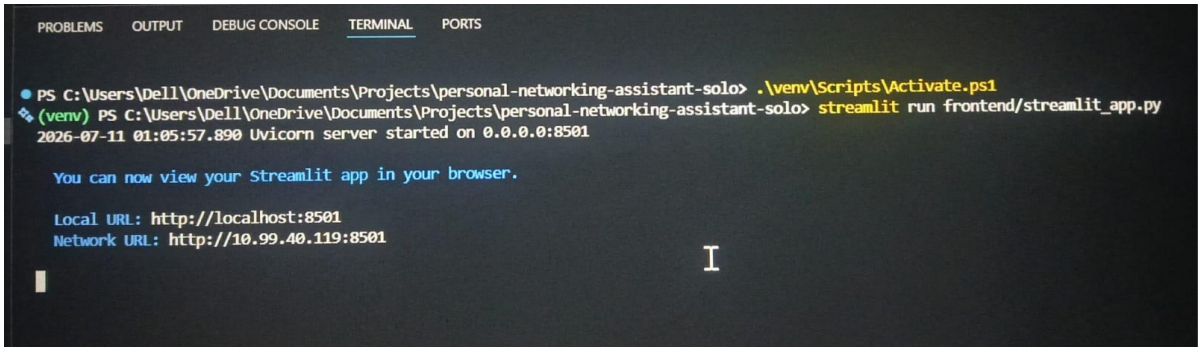
```
uvicorn app.main:app –reload

streamlit run frontend/streamlit_app.py
```

- Open a new terminal and start the Streamlit frontend:
  `streamlit run frontend/streamlit_app.py`

- Verify that both services are running successfully before testing the application.





Application Verification:

- Verify that both the FastAPI backend and Streamlit frontend are running successfully.
- Open the following URLs in your browser to test the application:

```
Backend API: http://127.0.0.1:8000
```

```
Frontend UI: http://localhost:8501
```

- Ensure that the FastAPI backend and Streamlit frontend are configured correctly before running the application.
- Verify that all required dependencies are installed and the virtual environment is activated.

Run the Application:

- Start the FastAPI backend and Streamlit frontend using the following commands:

```
uvicorn app.main:app –reload

streamlit run frontend/streamlit_app.py
```

- After both services start successfully, the Personalized Networking Assistant can be accessed locally through the browser.
- Use the frontend interface to enter event details and generate networking suggestions.

Important Notes:

• Ensure that both FastAPI and Streamlit services are running simultaneously.

• The backend service runs on http://127.0.0.1:8000.

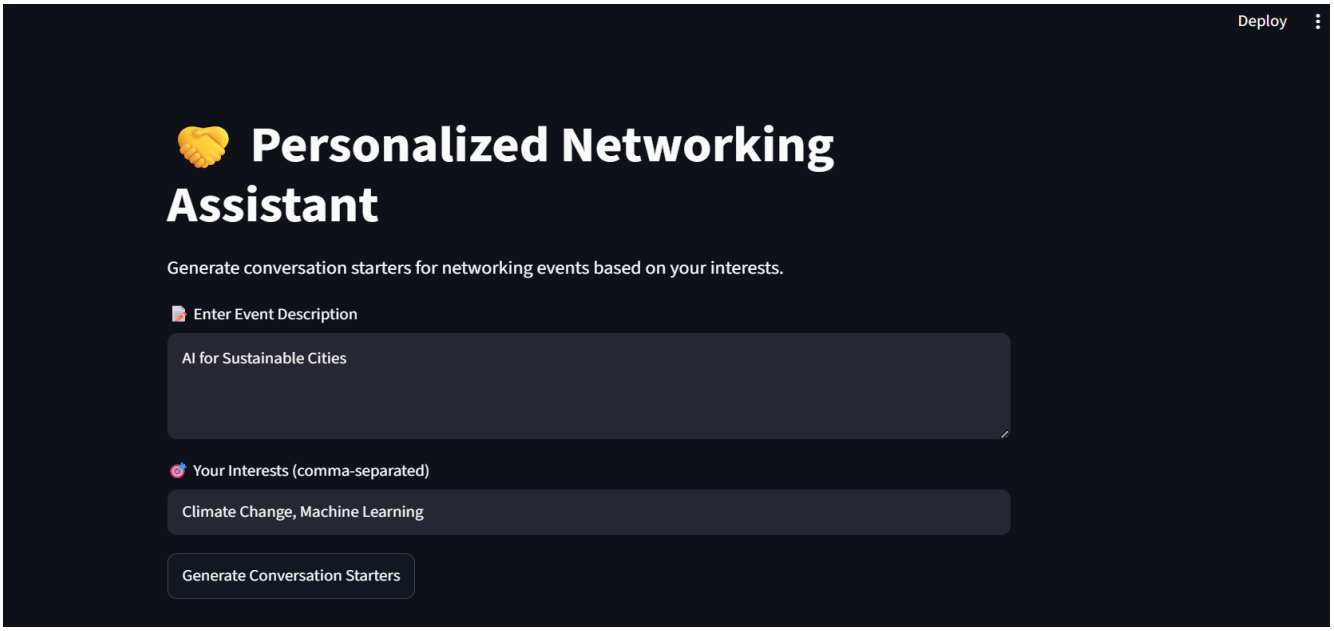
• The frontend service runs on http://localhost:8501.

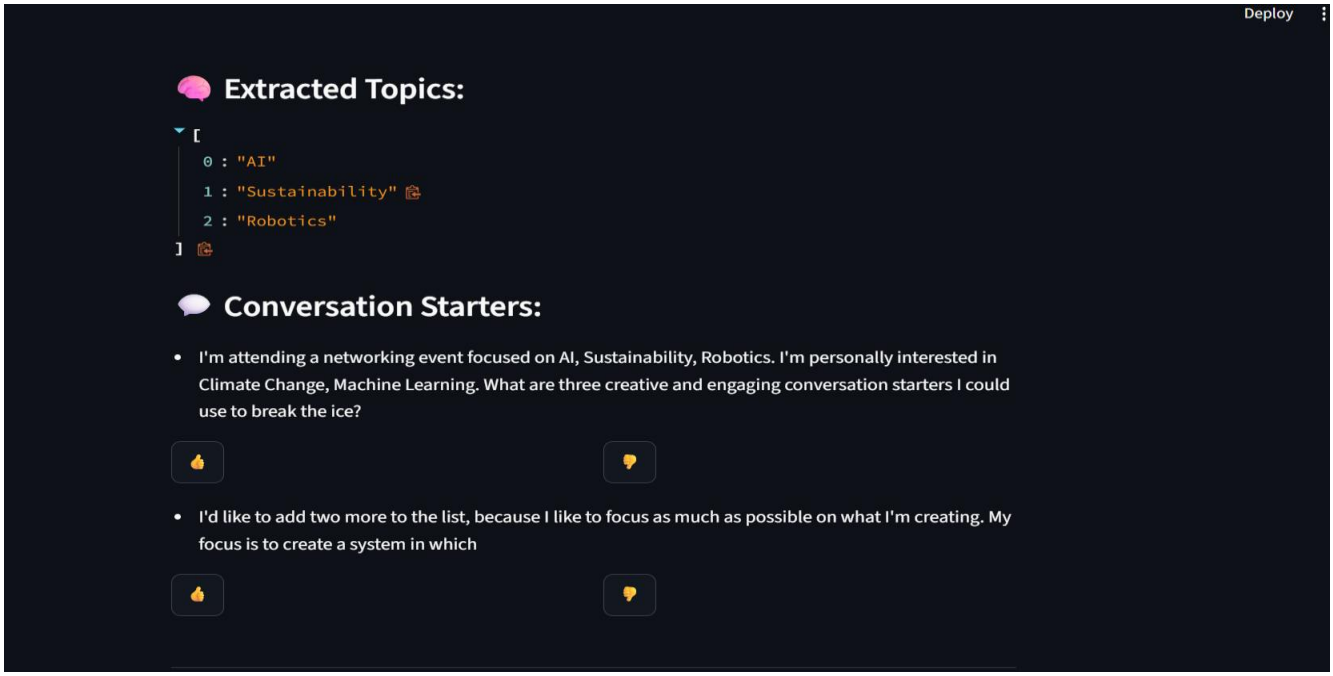
• Stop both services using CTRL + C after testing.

• Install missing dependencies using requirements.txt if any error occurs.

Exploring the Personalized Networking Assistant:

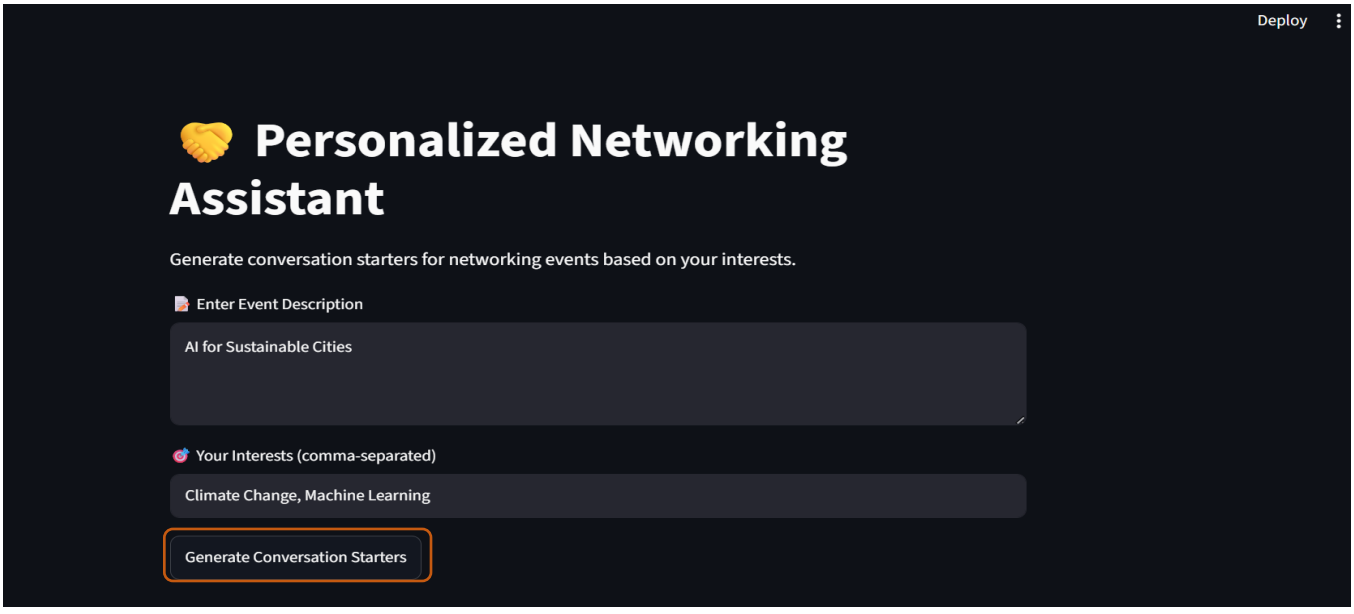
Event Analysis and Conversation Generator Page:





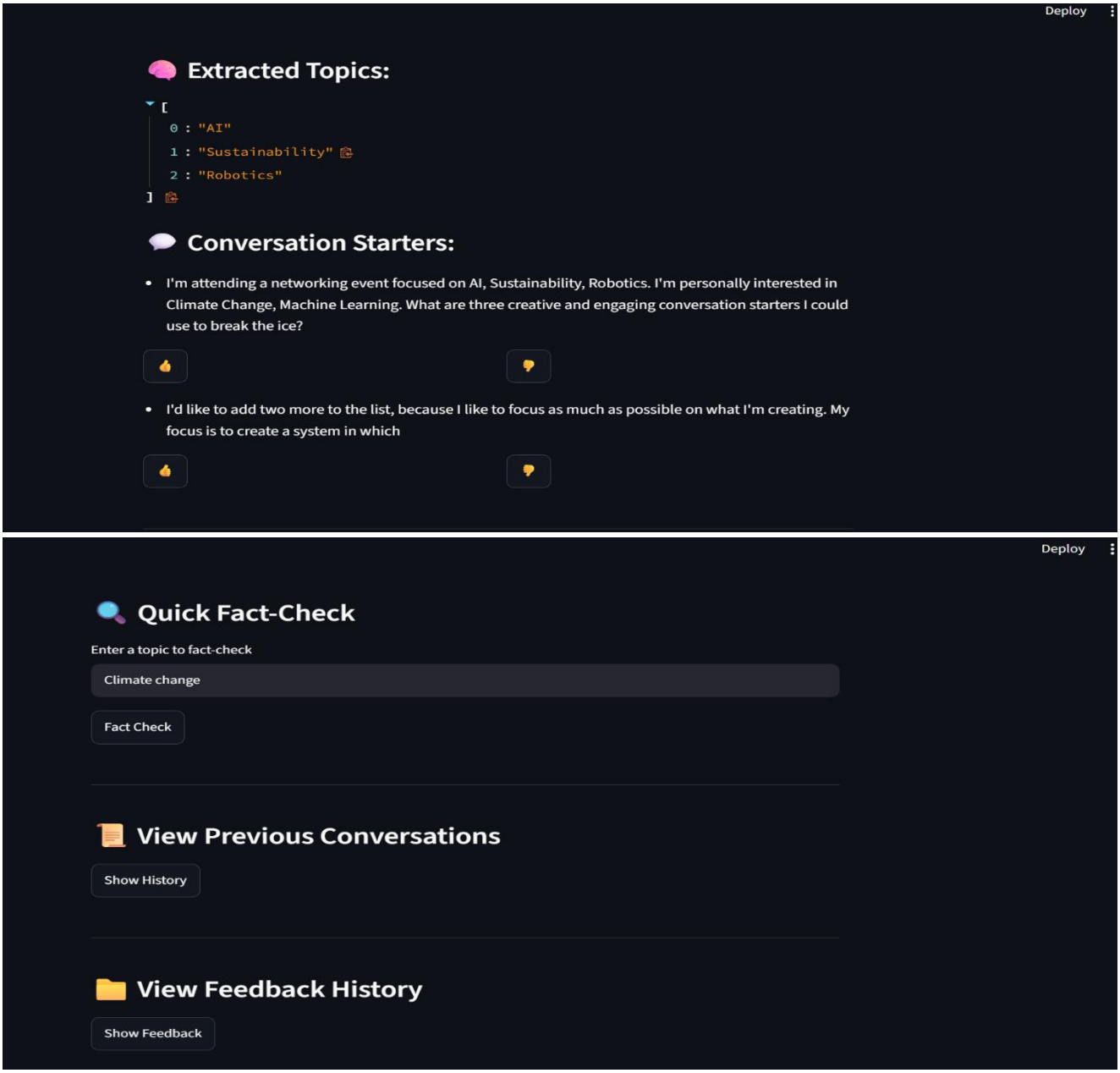
Description: The Personalized Networking Assistant is a Streamlit-based web application that helps users generate context-aware conversation starters for networking events. Users can enter an event description and their interests, and the system generates personalized suggestions using AI models integrated with the FastAPI backend.

Event and Interest Input Section:



Description: The input section allows users to enter an event description and their interests. Users provide details about the networking event and their preferred topics, then click the "Generate Conversation Starters" button to receive AI-generated conversation suggestions.

Results Section:



Description: After the user clicks the "Generate Conversation Starters" button, the application displays the extracted topics and AI-generated conversation suggestions. Users can view multiple conversation starters and provide feedback using the like and dislike buttons.

Conclusion:

The Personalized Networking Assistant is an AI-powered web application developed using FastAPI and Streamlit. It helps users generate personalized conversation starters for networking events based on event descriptions and user interests. The project integrates AI models for topic extraction, conversation generation, fact-checking, and feedback collection.

The application demonstrates how artificial intelligence can improve networking experiences by providing relevant and engaging conversation suggestions. Features such as topic extraction, fact-checking, conversation history, and feedback management make the platform practical and user-friendly.

In the future, the project can be enhanced by improving the AI models, adding support for more data sources, and expanding personalization features. Overall, the Personalized Networking Assistant showcases the potential of AI-powered tools in simplifying communication and professional networking.